

Revisão de LPOO

Programação Orientada a Objetos em Java — baseada no FanVerse e no Java-activities

Cotação de 15 de setembro de 2026 — preços em USD, mercado dos EUA.

Preços de hardware mudam diariamente: confira o link da fonte antes de comprar.

Projetos-Futuros / base-movel

LPOO — Revisão Completa Baseada no Seu Próprio Código

Material montado a partir da leitura de **dois repositórios seus**: FanVerse (commit 8each09) e Java-activities (commit 7e57bea). Todo exemplo abaixo é **código que você escreveu** — não exemplo genérico de livro.

1. Inventário: o que você já tem escrito

	FanVerse	Java-activities	Total
Arquivos .java	12	91	103
Atividades	—	12 (ATV1 a ATV12) + 2 projetos	—

Conceitos medidos no código (nº de arquivos que usam)

Conceito	Arquivos	Situação
public	91	✓ Dominado
private	59	✓ Encapsulamento é hábito seu
@Override	26	✓ Sobrescrita bem usada
extends	19	✓ Herança sólida
super(...)	19	✓ Encadeia construtor corretamente
static	25	✓
final	15	✓
super. (método)	10	✓
throw	8	! Lança, mas nunca captura
List< / ArrayList	5 / 4	[~] Uso básico
implements	4	[~] Pouco
interface	1	[~] Só a Alugavel
protected	1	[~] Quase não usa
Map<	1	[~]
abstract	0	✗ Nunca usou
try / catch	0	✗ Nunca usou
enum	0	✗
instanceof	0	✗
equals / hashCode	0	✗ Causa um bug real — ver §7

Leitura honesta: você domina os três primeiros pilares (encapsulamento, herança, polimorfismo) com folga. O buraco está em **abstração formal** (abstract/interface), **tratamento de exceção** e **contrato de igualdade**. É exatamente aí que este material aperta.

2. Os quatro pilares, no seu código

2.1 Encapsulamento — você já faz certo

Atributo privado + getter/setter **com validação**. De FanVerse/src/livros/Livro.java:

```
private String titulo;

public void setTitulo(String titulo) {
    if (titulo == null || titulo.trim().isEmpty()) {
        throw new IllegalArgumentException("O título do livro não pode ficar vazio.");
    }
    this.titulo = titulo.trim();
}
```

Por que isso é encapsulamento de verdade e não "getter/setter burro": o setter protege um invariante — nenhum Livro do sistema pode existir com título vazio. Um setter que só faz `this.x = x` não encapsula nada, apenas torna o campo público com passos extras.

Detalhe que o professor valoriza: seu construtor chama os setters, não atribui direto:

```
public Livro(String titulo, String autor, int ano) {
    setTitulo(titulo); // ← validação roda também na construção
    setAutor(autor);
    setAno(ano);
    this.codigoLivro = new CodigoLivro();
}
```

Assim a validação vale na criação e na alteração, sem duplicar código.

! Contraponto teórico que pode cair na prova: chamar método sobrescrevível dentro do construtor é considerado má prática, porque uma subclasse pode sobrescrever o setter e recebê-lo executando antes de seus próprios campos existirem. A correção é declarar os setters como final (ou private) quando eles são chamados no construtor.

Cuidado de encapsulamento que você já acertou — `Colecao.getLivros()`:

```
public List<Livro> getLivros() {
    return Collections.unmodifiableList(livros);
}
```

Retornar a List direto deixaria qualquer um chamar `colecacao.getLivros().clear()` e furar o encapsulamento. `unmodifiableList` fecha essa porta. **Isso é nível acima do que a maioria entrega.**

2.2 Herança — extends + super

De FanVerse/src/livros/LivroDigital.java:

```
public class LivroDigital extends Livro {
    private double tamanhoArquivo;

    public LivroDigital(String titulo, String autor, int ano, double tamanhoArquivo) {
        super(titulo, autor, ano); // ← PRIMEIRA linha, obrigatoriamente
        setTamanhoArquivo(tamanhoArquivo);
    }
}
```

Três regras de prova:

1. `super(...)` tem que ser a **primeira instrução** do construtor.
2. Se você **não** escrever `super(...)`, o Java insere `super()` implícito. Se a superclasse **não tiver** construtor sem argumentos → **erro de compilação**.
3. Java **não tem herança múltipla de classes**. Uma classe estende uma só. Para "herdar" de várias fontes, usa-se interface.

2.3 Polimorfismo — as duas formas

Sobrescrita (overriding) — mesma assinatura, comportamento diferente. De Livro e LivroDigital:

```
// Livro.java
public void apresentarDados() {
    exibirDadosBasicos();
}

// LivroDigital.java
@Override
public void apresentarDados() {
    exibirDadosBasicos();
    System.out.println("Tipo: Livro Digital");
    System.out.println("Tamanho do arquivo: " + tamanhoArquivo + " MB");
}
```

Isso permite o polimorfismo em ação:

```
Livro l = new LivroDigital("Titulo", "Autor", 2024, 5.2);
l.apresentarDados(); // chama a versão de LivroDigital
```

A pergunta clássica de prova: o tipo da variável é *Livro*, mas o método executado é o de *LivroDigital*. Isso se chama **ligação tardia** (late binding) — quem decide é o tipo do **objeto** em tempo de execução, não o da variável.

Sobrecarga (overloading) — mesmo nome, parâmetros diferentes. Você já usa, em Livro.java:

```
public void exibirMensagem() { ... }
public void exibirMensagem(String mensagem) { ... }
```

	Sobrescrita (override)	Sobrecarga (overload)
Onde	Entre classe mãe e filha	Na mesma classe (ou herdada)
Assinatura	Idêntica	Diferente (tipo/qtd de parâmetros)
Decidido em	Execução	Compilação
Tipo de retorno	Igual (ou covariante)	Pode mudar
@Override	Sim	Não

Mudar **só o tipo de retorno** não é sobrecarga — é erro de compilação.

2.4 Abstração — ****o seu buraco nº 1****

abstract aparece em **zero** dos seus 103 arquivos. E há um caso onde ela deveria estar. De Java-activities/ATV 11/DELTA/Forma.java:

```
public class Forma {
    public double calcularArea() {
        return 0.0;
    }
}
```

// ← deveria ser abstract
// ← "área zero" não existe

O problema concreto: new Forma("qualquer") compila e devolve um objeto com área 0. **Uma "forma genérica" não é uma coisa que existe no mundo.** Corrigido:

```
public abstract class Forma {
    private String nome;

    public Forma(String nome) { this.nome = nome; }

    public abstract double calcularArea(); // sem corpo: obriga a subclasse

    public void apresentarDados() {
        System.out.println("Forma: " + nome);
        System.out.println("Area: " + calcularArea()); // polimorfismo
    }
}
```

O que isso ganha:

Antes	Depois
new Forma(...) compila	new Forma(...) → erro de compilação
Subclasse pode esquecer de sobrescrever	Compilador obriga
calcularArea() mente (retorna 0.0)	Não existe implementação falsa

O mesmo vale para ATV 11/OMEGA/Animal.java e ATV 12/NOVEMBER/Animal.java: emitirSom() imprimindo "O animal emitiu um som" é o mesmo sintoma.

3. Classe abstrata vs. interface

Você tem **1 interface** no total — Alugavel, em projeto_analise_algoritmos:

```
public interface Alugavel {
    double calcularValorDiaria();
    boolean isDisponivel();
    String getDescricao();
}
```

E Carro implements Alugavel. Isso está **certo** e é o seu melhor código de abstração. Só está subutilizado.

A tabela que cai em prova

	abstract class	interface
Herança múltipla	X Uma só	✓ Várias (implements A, B, C)
Atributos de instância	✓ Sim	X Só public static final
Construtor	✓ Sim	X Não
Métodos com corpo	✓ Sim	✓ Só default / static (Java 8+)
Métodos sem corpo	✓ abstract	✓ (padrão)
Modificadores	Qualquer	Métodos são public implicitamente
Instanciar direto	X	X

Como escolher

- **"É UM"** com estado e código compartilhado → **classe abstrata**. LivroDigital **é um** Livro.
- **"CONSEGUE FAZER"** / capacidade → **interface**. Carro **consegue ser** alugado (Alugavel).

Uma classe pode fazer as duas coisas:

```
public class Carro extends Veiculo implements Alugavel, Comparable<Carro> {
```

Onde aplicar no FanVerse

Livro é a candidata perfeita:

```
public abstract class Livro {
    public abstract double calcularPreco(); // digital e físico cobram diferente
    public abstract String getFormato();
}
```

E uma interface para o que não é hierarquia:

```
public interface Emprestavel {
    boolean estaDisponivel();
    void emprestar(Usuario u);
    void devolver();
}
```

LivroFísico implements Emprestavel — mas LivroDigital não, porque e-book não é emprestado do mesmo jeito. **Interface separa capacidade de hierarquia.**

4. Composição — você usa e talvez não saiba o nome

De Livro.java:

```
private CodigoLivro codigoLivro; // Livro TEM-UM CodigoLivro
```

De Colecao.java:

```
private List<Livro> livros; // Colecao TEM-VÁRIOS Livro
```

Relação	Palavra-chave	Exemplo seu
É UM (herança)	extends	LivroDigital extends Livro
TEM UM (composição)	atributo	Livro tem CodigoLivro

A regra que o mercado usa: prefira composição a herança. Herança acopla forte — mudar a mãe quebra todas as filhas. O teste: se você não consegue dizer "X **é um** Y" em voz alta sem soar estranho, use composição.

5. static e final

Você usa os dois corretamente. De FanVerse/src/compras/Compra.java:

```
private static int contador = 1; // compartilhado por TODAS as instâncias
```

De Locadora/Locadora.java:

```
private static final double[] TABELA_PRECOS = {90.0, 130.0, 180.0};
```

Modificador	Em atributo	Em método	Em classe
static	Um valor para toda a classe	Chamado sem objeto	Só classe aninhada
final	Constante, não reatribui	Não pode sobrescrever	Não pode herdar

! Pegadinha clássica: final num objeto trava a **referência**, não o conteúdo. `private final List<Livro> livros = new ArrayList<>();` ainda aceita `livros.add(...)`. O que não dá é `livros = outraLista;`. Seu `TABELA_PRECOS` é final mas os valores do array **podem** ser trocados — `TABELA_PRECOS[0] = 999;` compila.

6. Exceções — seu buraco nº 2

Você tem **8 arquivos que lançam** exceção e **zero que capturam**:

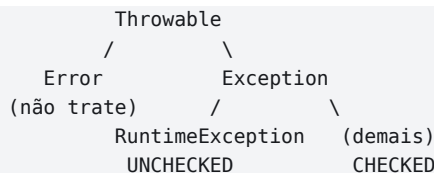
```
// Livro.java – você lança
throw new IllegalArgumentException("0 título do livro não pode ficar vazio.");
```

Mas em nenhum lugar existe try/catch. Na prática, um título vazio **derruba o programa** com stack trace.

O que falta

```
try {
    Livro l = new Livro("", "Autor", 2024);
} catch (IllegalArgumentException e) {
    System.out.println("Erro ao criar livro: " + e.getMessage());
} finally {
    System.out.println("Sempre executa – ideal para fechar recurso.");
}
```

A hierarquia (cai em prova)



	Checked	Unchecked
Exemplos	IOException, SQLException	IllegalArgumentException, NullPointerException
Herda de	Exception	RuntimeException
Compilador exige tratar	✓ Sim	✗ Não
Significa	Falha externa previsível	Erro de programação

Suas IllegalArgumentException e IndexOutOfBoundsException são **unchecked** — por isso compilam sem try/catch. Está correto: validação de argumento é unchecked por convenção. Mas o Principal deveria capturá-las para o programa não morrer.

Ordem dos catch: do **mais específico para o mais genérico**. Se catch (Exception e) vier primeiro, os seguintes viram código inalcançável e não compilam.

7. equals e hashCode — um bug real no seu código

Zero ocorrências nos dois repositórios. Isso não é só teoria: cria um bug silencioso em Colecao.java.

```
public boolean removerLivro(Livro livro) {
    return livros.remove(livro);    // usa equals() internamente
}
```

Sem equals() sobrescrito, o Java usa o de Object, que compara **endereço de memória**. Logo:

```
Colecao c = new Colecao("Serie", "desc", "autor");
c.adicionarLivro(new Livro("Duna", "Herbert", 1965));

// tenta remover um livro "igual"
boolean removeu = c.removerLivro(new Livro("Duna", "Herbert", 1965));
// removeu == false ← e a coleção continua com 1 livro
```

Dois objetos com exatamente os mesmos dados não são considerados iguais. O método falha em silêncio, sem exceção, e o `removerLivro` só funciona se você passar a mesma referência que adicionou.

A correção

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    Livro outro = (Livro) o;
    return ano == outro.ano
        && Objects.equals(titulo, outro.titulo)
        && Objects.equals(autor, outro.autor);
}

@Override
public int hashCode() {
    return Objects.hash(titulo, autor, ano);
}
```

O contrato (isto cai em prova)

1. Se `a.equals(b)` é true, então `a.hashCode() == b.hashCode()` **é obrigatório**.
2. A recíproca **não** vale: hash igual não obriga equals igual (colisão).
3. **Sobrescreveu equals sem hashCode → quebra HashMap e HashSet.** O objeto "some" dentro da coleção: você insere e `contains()` devolve false.

Este é, em conteúdo de prova, o assunto que mais separa quem estudou de quem decorou. E no seu caso tem consequência prática imediata.

8. Modificadores de acesso

Você usa `protected` em **1 arquivo só** — e o uso está certo. De `Livro.java`:

```
protected void exibirDadosBasicos() {
    System.out.println("Título: " + titulo);
    // ...
}
```

É exatamente o caso de uso: método que **as filhas usam** (`LivroDigital` e `LivroFisico` chamam) mas que **não é API pública** da classe.

Modificador	Mesma classe	Mesmo pacote	Subclasse (outro pacote)	Qualquer um
private	✓	✗	✗	✗
(padrão)	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

Regra de ouro: comece tudo `private`. Só abra quando houver necessidade real. Campo `public` em prova de LPOO é desconto de nota quase automático.

9. O que falta no seu repertório

9.1 enum — zero usos

Você tem status de compra como texto. Com enum fica seguro:

```
public enum StatusCompra {
    PENDENTE("Aguardando pagamento"),
    PAGA("Pagamento confirmado"),
    CANCELADA("Compra cancelada");

    private final String descricao;

    StatusCompra(String descricao) { this.descricao = descricao; }

    public String getDescricao() { return descricao; }
}
```

Ganho: String aceita "pgaa" com erro de digitação; enum **não compila** se o valor não existir.

9.2 instanceof e pattern matching — zero usos

```
// Forma antiga
if (livro instanceof LivroDigital) {
    LivroDigital ld = (LivroDigital) livro;
    ld.baixar();
}

// Java 16+ – pattern matching, mais limpo
if (livro instanceof LivroDigital ld) {
    ld.baixar();
}
```

*! Mas: **muito instanceof é sinal de polimorfismo mal feito**. Se você está testando o tipo para decidir o comportamento, provavelmente o comportamento deveria ser um método sobrescrito.*

9.3 Coleções além de ArrayList

Interface	Implementação	Quando usar
List	ArrayList	Ordem importa, aceita repetido
Set	HashSet	Sem repetição (precisa de equals/hashCode!)
Map	HashMap	Busca por chave — Map<String, Livro> por ISBN
Queue	LinkedList	Fila de espera de empréstimo

Seu Colecao.procurarLivroPorTitulo() percorre a lista inteira — O(n). Com Map<String, Livro> viraria O(1).

10. Mapa: onde cada conceito está no seu código

Conceito	Arquivo	Repositório
Encapsulamento com validação	src/livros/Livro.java	FanVerse
Herança + super(...)	src/livros/LivroDigital.java	FanVerse
Sobrescrita (@Override)	src/livros/LivroFisico.java	FanVerse

Sobrecarga	Livro.exibirMensagem()	FanVerse
Composição	Livro → CodigoLivro	FanVerse
Coleção encapsulada	Colecao.getLivros()	FanVerse
Depreciação e migração	biblioteca/Compra.java (@Deprecated)	FanVerse
Interface	locadora/Alugavel.java	Java-activities
static final (constante)	locadora/Locadora.java	Java-activities
Campos final	locadora/Carro.java	Java-activities
Hierarquia de funcionários	ATV 6/ALFA/	Java-activities
Hierarquia de contas	ATV 10/GOLF/	Java-activities
Polimorfismo com formas	ATV 11/DELTA/	Java-activities
Polimorfismo com animais	ATV 11/OMEGA/, ATV 12/NOVEMBER/	Java-activities
Camadas (model/service/controller)	Java_Outras_Coisas/biblioteca/	Java-activities

Detalhe que merece crédito: em biblioteca/Compra.java você marcou a classe antiga como @Deprecated e fez ela estender a nova (compras.Compra) em vez de apagar. Isso é **estratégia de migração sem quebrar código existente** — não é conteúdo de primeiro semestre.

11. Questões típicas de prova, com resposta

- 1. Diferença entre sobrecarga e sobrescrita?** Sobrecarga: mesmo nome, **parâmetros diferentes**, mesma classe, resolvido em **compilação**. Sobrescrita: **mesma assinatura**, classe filha, resolvido em **execução**.
- 2. Pode instanciar classe abstrata?** Não. Mas pode ter construtor (chamado via super() pela filha) e pode ter referência do tipo abstrato apontando para objeto concreto.
- 3. Classe abstrata pode não ter método abstrato?** Pode. Basta abstract na declaração para impedir instanciação.
- 4. Interface pode ter método com corpo?** Desde o Java 8, sim: default e static. Desde o 9, também private.
- 5. Para que serve super?** Duas coisas: super(...) chama o construtor da mãe (1ª linha); super.metodo() chama a versão da mãe de um método sobrescrito.
- 6. O que acontece se eu sobrescrever equals sem hashCode?** Quebra HashMap/HashSet: o objeto vai para um balde diferente e some. **Contrato violado**.
- 7. final em classe, método e atributo?** Classe: não pode ser herdada. Método: não pode ser sobrescrito. Atributo: não pode ser reatribuído (mas o objeto apontado ainda muda por dentro).
- 8. Java tem herança múltipla?** De **classe**, não. De **interface**, sim (implements A, B). Evita o problema do diamante.
- 9. Diferença entre checked e unchecked?** Checked herda de Exception: o compilador **obriga** tratar. Unchecked herda de RuntimeException: erro de programação, não obriga.
- 10. O que é ligação tardia?** A decisão de qual versão do método executar acontece em **tempo de execução**, pelo tipo do objeto, não pelo tipo da variável. É o motor do polimorfismo.

12. Plano de estudo — o caminho mais curto

Na ordem de retorno por hora investida:

1. **abstract** (1 h) — transforme Forma e Animal em abstratas nas ATV 11 e 12. Você já tem o código; é trocar 3 linhas e sentir o compilador cobrar.
2. **equals/hashCode** (1 h) — implemente em Livro e escreva o teste que remove um livro "igual" da Colecao. Veja falhar antes, passar depois.
3. **try/catch** (1 h) — envolva as chamadas do Principal do FanVerse. É o pilar que falta completo.
4. **Interface** (1 h) — crie Emprestavel no FanVerse e faça só LivroFisico implementar. Sinta a diferença entre hierarquia e capacidade.
5. **enum** (30 min) — troque o status da Compra por enum.

Depois desses cinco, não sobra buraco conceitual de LPOO no seu código.

13. Leitura de fonte

Assunto	Fonte
Tutorial oficial de OO	Oracle — Object-Oriented Programming Concepts
Classes abstratas	Oracle — Abstract Methods and Classes
Interfaces	Oracle — Interfaces
Contrato de equals	Javadoc — Object.equals)
Exceções	Oracle — Exceptions
Coleções	Oracle — Collections Framework